

# Evaluating different optimizations on ZPIC using heterogeneous architecture at Deucalion

1<sup>st</sup> Diogo Coelho da Silva  
University of Minho  
Braga, Portugal  
pg61444@alunos.uminho.pt

2<sup>nd</sup> Tomás Alexandre Torres Pereira  
University of Minho  
Braga, Portugal  
pg59810@alunos.uminho.pt

**Abstract**—Particle-In-Cell (PIC) simulations transition between compute-bound and memory-bound regimes depending on problem size, with large grids dominated by memory bandwidth due to stencil-based current smoothing and Maxwell solvers.

This work evaluates the performance of ZPIC on the Deucalion supercomputer under two configurations. In the laser-plasma case, OpenMP on the Fujitsu A64FX shows that data-layout transformations and SVE vectorisation substantially improve single-core performance, while scalability is ultimately limited by HBM2 bandwidth saturation. In the two-stream instability case, execution is dominated by the particle pusher (`spec_advance`). Hybrid MPI+OpenMP runs on A64FX and AMD EPYC 7742 enable multi-node execution but scale poorly due to particle redistribution and collective communication overheads. CUDA acceleration on an NVIDIA A100 yields speedups for moderate problem sizes, yet performance is constrained by CPU-GPU data transfers and atomic contention during current deposition.

**Index Terms**—ZPIC, Memory-bound, Simulation, SVE, Deucalion, NVIDIA A100, Fujitsu A64FX, AMD EPYC 7742

## I. INTRODUCTION

Particle-In-Cell (PIC) simulations model the interaction of charged particles with electromagnetic fields on a grid, exposing parallelism but often becoming memory-bound due to irregular particle-driven memory accesses [1]. As a result, performance degrades for large-scale simulations when relying solely on single-node optimizations.

This work studies ZPIC, an educational PIC solver, analysing its architecture and identifying optimization opportunities using performance-driven metrics. Experiments were conducted on the Deucalion supercomputer, exploiting its heterogeneous architectures.

We evaluate data layout transformations, vectorization, and OpenMP on the Fujitsu A64FX, extend the analysis to MPI-based distributed execution on both CPU platforms, and assess GPU acceleration using CUDA on an NVIDIA A100. The study is structured around two simulation configurations, each analysed independently due to distinct performance bottlenecks arising from their input parameters.

## II. METHODOLOGY

To ensure a consistent approach in exploiting various types of optimizations, we applied a strict and systematic methodology throughout the identification process. We used the following process, which is subdivided into four stages:

- Comprehensive study and understanding of the particle-in-cell simulation code.
- Conducting a hotspot analysis to identify bottleneck-critical regions.
- Implementing the targeted optimizations within the hotspot and general findings.
- Analyzing the results and validating them according to the architecture used.

## III. PARTICLE-IN-CELL METHOD

The Particle-In-Cell (PIC) method solves a coupled system of partial differential equations describing the self-consistent interaction between charged particles and electromagnetic fields. In this work, ZPIC adopts a one-dimensional spatial grid, restricting particle motion to a single coordinate while retaining full vector field components.

The algorithm can be viewed as a pipeline of dependent stages executed sequentially, including particle pushing, charge deposition, and the solution of Maxwell's equations. Because realistic simulations involve an extremely large number of particles, macro-particles are used to represent the collective behaviour of many physical particles, reducing computational cost while preserving accuracy.

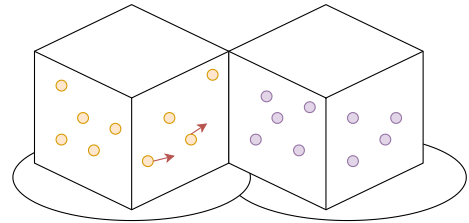


Fig. 1. One-dimensional Particle-In-Cell grid

Among these stages, the particle pusher is typically the most computationally intensive, as it processes all particles and their count far exceeds the number of grid cells. For this reason, the second simulation focuses on accelerating this phase through multi-node parallelization and CUDA-based GPU offloading, addressing the performance limitations observed in single-node executions.

#### IV. WA1: LASER PARTICLES SIMULATION

In the first work assignment, the particle-in-cell simulation uses the file `laser_particles.c` as input. The table below outlines the general simulation parameters defined within this file:

TABLE I  
SPECIFICATIONS OF SIMULATION USED IN FIRST ASSIGNMENT

|                                 |                 |
|---------------------------------|-----------------|
| Simulation name                 | Laser particles |
| N° of grid cells                | 20000           |
| N° of species                   | 1 ('electrons') |
| Box cell size                   | 20 units        |
| N° of particles per cell        | 10              |
| Total simulation time           | 40 time units   |
| Simulation time increments (dt) | 0.00095         |

The configurations above define a simulation with a large grid and a low particle-per-cell count. Consequently, grid-based operations, such as the Maxwell solver and current smoothing, are expected to represent a larger share of the total execution time.

In this simulation, we conducted vectorization tests and a strong scalability analysis in order to find bottlenecks and future improvements. Before optimizing the code, we performed a hotspot analysis to assess which bottlenecks are consuming the most resources during the simulation.

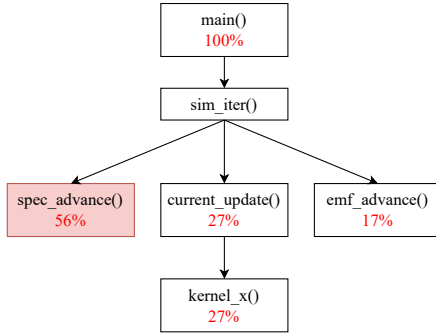


Fig. 2. This figure illustrates the hotspots of ZPIC at single-core level. The data was retrieved from Firefox Profiler.

As illustrated in Figure 2, our simulation exhibits three primary bottlenecks, each corresponding to one of the main phases of the particle-in-cell algorithm. Consistent with our earlier predictions, the Boris pusher emerges as the dominant bottleneck, consuming approximately 56% of the total simulation time.

##### A. Single-core optimizations and the effect of SVE vectorization on A64FX performance

During the Roofline analysis, we observed that the current kernel did not execute any SVE instructions. Given that leveraging vector instructions is crucial for achieving high performance on the A64FX architecture, we focused on optimizing the data structures within the identified hotspots to

enable effective vectorization. To enable SVE vectorization, we used the Clang compiler instead of GCC, as Clang provides more effective support for enforcing SVE optimizations.

On all the hotspots, we identified that data structures like `Float3` and `Particle` were in an Array of Structures format (AoS), which were limiting the vectorization. To address this limitation, we reorganized the data into a Structure of Arrays (SoA) layout, which exposes contiguous vectors of each field and therefore enables much more effective vectorization. We also add aligned allocation, using 256-byte aligned data, which corresponds to L1D and L2 cache line size.

As expected, the applied optimizations led to an increase in the number of executed SVE instructions, which realistically results in the use of both FLA and FLB pipelines. These optimizations resulted in a decrease of 82.7 seconds to 39.4 seconds, culminating in a  $2.1\times$  speedup in the single-core configuration. Despite this improvement, the primary limitation to further vectorization was the difficulty of optimizing `spec_advance()`, which remains the dominant hotspot in the application.

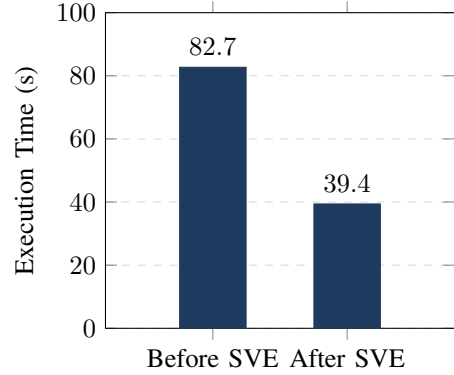


Fig. 3. This figure compares the execution time between a ZPIC using SVE pipelines and one without it.

##### B. Multi-core optimizations on A64FX

Although SVE vectorization significantly improves single-core performance, the Fujitsu A64FX achieves its highest efficiency through thread-level parallelism. We therefore analyse the impact of OpenMP parallelization on the main computational kernels.

A preliminary hotspot analysis confirms that, even after vectorization, `spec_advance` remains the dominant kernel due to irregular memory accesses during interpolation and current deposition, which also limit vectorization opportunities. OpenMP parallelization was introduced on the main particle loop. To avoid atomic contention during current deposition, the domain was decomposed into independent tiles of 256 cells, eliminating synchronization and preparing the code for future MPI extensions.

Additional optimizations were applied to `kernel_x`, which performs a memory-bound 3-point stencil for current smoothing. To avoid data races, computation and write-back were decoupled using a temporary buffer. The electromagnetic field

update routines, `yee_e()` and `yee_b()`, were also parallelized, but their very low arithmetic intensity ( $\approx 0.23$  FLOP/byte) makes them strictly memory-bandwidth bound.

As shown in the scalability and speedup results (Appendix A), performance scales up to 12 threads, corresponding to a single CMG, after which scalability degrades. Beyond this point, OpenMP overhead increases and execution becomes dominated by HBM2 bandwidth [2] contention. A maximum speedup of  $4.31\times$  is achieved at 12 threads relative to the SVE-optimized baseline (Figure 11, Appendix A), confirming that ZPIC is strongly memory-bound on A64FX. Increasing parallelism beyond HBM2 saturation yields no further gains.

## V. WA2: TWO STREAM SIMULATION

In the second work assignment, the particle-in-cell simulation is configured using the input file `twostream.c`. Similar to the first simulation, the table below presents the general simulation parameters specified in this file.

TABLE II  
SPECIFICATIONS OF SIMULATION USED IN FIRST ASSIGNMENT

|                                 |                                 |
|---------------------------------|---------------------------------|
| Simulation name                 | Laser particles                 |
| N° of grid cells                | 120                             |
| N° of species                   | 2 ('electrons', right and left) |
| Box cell size                   | 4*PI units                      |
| N° of particles per cell        | 500                             |
| Total simulation time           | 50 time units                   |
| Simulation time increments (dt) | 0.1                             |

Unlike the previous simulation, this configuration employs a smaller grid, resulting in an accentuated hotspot during the particle push phases.

In order to determine the primary performance bottlenecks, we conducted a hotspot analysis on the original ZPIC codebase. The analysis indicated that nearly 90% of the runtime is concentrated in the `spec_advance` function. Motivated by this result, we developed a GPU implementation for the NVIDIA A100 as well as an MPI-based implementation targeting both CPU architectures.

### A. Do the changes in the simulation affect parallelism?

#### 1) A64FX Strong Scalability and Speedup

The strong scalability of the Fujitsu A64FX for the WA2 configuration is summarized in Figure 12. Execution time decreases rapidly up to 12 threads, corresponding to a single CMG, where scaling is close to ideal and OpenMP overhead is minimal.

Beyond this point, performance continues to improve but with diminishing returns as execution spans multiple CMGs. The particle-dominated nature of WA2 delays memory-bandwidth saturation compared to WA1, although shared-memory bandwidth and inter-CMG effects ultimately limit scalability.

The corresponding speedup follows the ideal linear trend at low thread counts and diverges at higher levels of parallelism, consistent with Amdahl's Law. Residual serial regions and memory contention bound performance at scale.

Overall, WA2 exhibits better strong scalability on the A64FX than WA1, effectively exploiting multiple CMGs before architectural limits are reached.

#### 2) AMD EPYC 7742 Strong Scalability and Speedup

The strong scalability results for the AMD EPYC 7742 under the WA2 configuration are reported in Figure 14. Execution time decreases steadily with increasing thread count, with substantial gains up to 64 threads, reflecting efficient exploitation of the architecture's high core count for the particle-dominated workload.

At higher thread counts, scalability tapers off due to NUMA effects and increased pressure on the memory hierarchy as execution spans multiple CCDs. The corresponding speedup follows the ideal trend at low to moderate concurrency levels and deviates at larger scales, consistent with Amdahl's Law and memory-access overheads.

Overall, the EPYC 7742 achieves higher peak speedup and lower execution time than the A64FX for WA2, highlighting its suitability for highly parallel particle-based simulations.

#### 3) Impact of the simulation changes: brief comparison

Compared to the laser-particle configuration (WA1), the two-stream simulation (WA2) significantly alters the computational balance of ZPIC. As shown in Figure 4, moving from WA1 to WA2 reduces execution time by over an order of magnitude, yielding an overall speedup of approximately  $62\times$  between the least and most favorable execution scenarios.

This improvement is driven by the shift in the dominant kernel. In WA2, execution is dominated by the particle pusher (`spec_advance`), which exhibits higher inherent parallelism than the grid-based kernels prevalent in WA1, enabling more effective scaling across threads, sockets, and nodes.

The baseline comparison also highlights architecture-dependent optimal configurations. While the A64FX achieves peak performance at relatively low thread counts in WA1, it benefits substantially from increased parallelism in WA2. The AMD EPYC 7742 attains the lowest execution times when fully exploiting its core count, reflecting the influence of cache hierarchy, NUMA topology, and memory bandwidth.

Overall, these results show that ZPIC scalability is strongly workload-dependent. The two-stream configuration exposes substantially greater parallelism, motivating the use of MPI-based execution and GPU acceleration in subsequent sections.

#### 4) Energy Analysis

Figure 5 shows the energy-to-solution of the ZPIC two-stream simulation as a function of thread count for both CPU architectures. Although increased parallelism reduces execution time on both platforms, their energy-efficiency trends differ substantially.

On the Fujitsu A64FX, energy-to-solution decreases sharply with increasing parallelism and reaches a minimum at moderate thread counts, after which it stabilizes, indicating a clear energetic sweet spot. In contrast, the AMD EPYC 7742 shows nearly flat energy consumption at moderate parallelism and increasing energy-to-solution at higher thread counts, reflecting higher power draw and NUMA-related overheads.

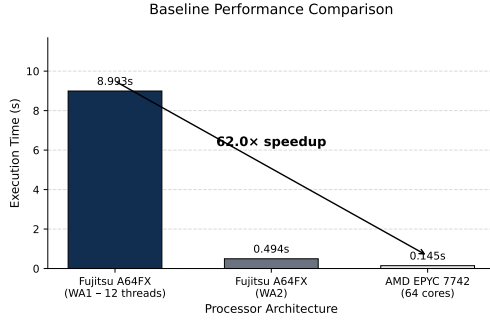


Fig. 4. Baseline performance comparison between the WA1 and WA2 configurations across architectures, using the optimal parallel setup for each platform. The arrow indicates the overall speedup (62 $\times$ ) from the slowest to the fastest execution.

Overall, while the EPYC 7742 achieves lower time-to-solution, the A64FX operates in a more energy-efficient regime for this workload. These results highlight the trade-off between performance and energy efficiency and motivate the exploration of accelerator-based solutions in the following section.

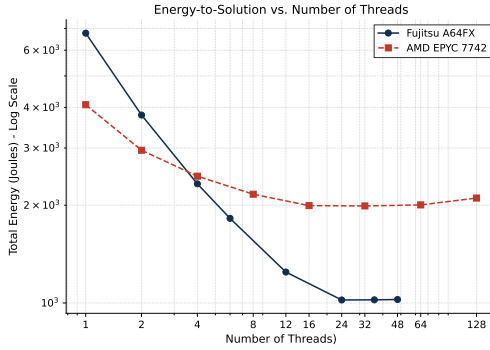


Fig. 5. Energy-to-solution of the ZPIC two-stream simulation as a function of the number of threads for the Fujitsu A64FX and AMD EPYC 7742, using logarithmic scales on both axes.

## B. The effect of using MPI on ZPIC

### 1) MPI Decomposition Strategy

To extend ZPIC beyond shared-memory scalability, an MPI-based implementation was developed targeting the dominant WA2 hotspot, the particle pusher (`spec_advance`), which accounts for nearly 90% of the execution time. Rather than refactoring the entire code, parallelization was deliberately confined to this phase.

The implementation adopts a particle-based decomposition, redistributing the global particle set across MPI ranks at each iteration using `MPI_Scatterv`. Each rank advances its local particles independently, performing field interpolation, Boris pushing, and local current deposition. OpenMP is used within each rank, yielding a hybrid MPI+OpenMP execution model with explicit thread affinity.

To preserve locality, MPI ranks were pinned to CMGs on the Fujitsu A64FX and to CCDs/NUMA domains on the AMD EPYC 7742, with OpenMP threads bound to cores within

each domain. To avoid atomic contention, current deposition is performed into private thread-local buffers, which are reduced locally within each rank.

After particle advancement, global consistency is restored through collective communication. Updated particle data are gathered using `MPI_Gatherv`, while current arrays and diagnostic quantities are combined with `MPI_Reduce` and broadcast to all ranks for the next iteration.

This approach preserves the original ZPIC structure with minimal refactoring, but scalability is limited by iteration-level particle redistribution and global collectives. Communication costs scale with the total particle count, and several phases remain serialized on the root rank, fundamentally constraining multi-node performance.

### 2) MPI Strong Scalability

Strong scalability of the MPI implementation was evaluated on the Fujitsu A64FX and AMD EPYC 7742 by fixing the global problem size and increasing the number of MPI ranks. Execution time results are reported in Appendix D (Figures 16 and 17).

On both architectures, only marginal performance improvements are observed at low rank counts, after which execution time increases as scaling extends across nodes. This behavior reflects the rapidly growing cost of iteration-level particle redistribution and collective communication, which outweighs the benefits of additional parallelism.

Overall, the results show that MPI strong scalability is fundamentally limited by communication rather than computation. Global synchronization and full particle redistribution at every timestep dominate execution time, bounding achievable speedup in accordance with Amdahl's Law.

### 3) MPI Weak Scalability

Weak scalability was evaluated by increasing the global problem size proportionally to the number of MPI ranks, keeping the number of particles per rank approximately constant. Results are shown in Appendix D (Figures 18 and 19).

Both architectures exhibit more stable behavior under weak scaling than under strong scaling, but execution time still increases with scale. On the Fujitsu A64FX, the gradual rise reflects balanced computation aided by CMG-aware rank placement. On the AMD EPYC 7742, execution time grows more sharply at higher core counts due to accumulating collective communication and synchronization overheads.

Overall, ideal weak scalability is not achieved on either platform, as communication costs grow with the number of MPI ranks and ultimately dominate execution time even when per-rank workload is held constant.

### 4) Discussion and Implications

The MPI implementation enables distributed-memory execution of ZPIC with minimal code restructuring and allows scalability studies on both CPU architectures. Architecture-aware rank placement and thread affinity are essential to avoid performance degradation due to poor memory locality.

However, scalability is fundamentally limited by algorithmic constraints. Full particle redistribution and synchronous global

reductions at every iteration dominate execution time at scale, severely restricting both strong and weak scalability.

Consequently, while MPI enables multi-node execution for moderate configurations, it is not an effective strategy for accelerating the WA2 hotspot. These results motivate GPU acceleration as the preferred approach, where the particle pusher can be executed on a single device with reduced communication and synchronization overheads, as explored in the following section.

### C. Using accelerators on ZPIC

After implementing MPI, we evaluated GPU acceleration using CUDA [3] on an NVIDIA A100. The targeted kernel is mostly compute-bound; however, the current deposition step introduces contention due to per-cell atomic operations.

As *spec\_advance.c* dominates execution time, it was the sole focus of the CUDA optimization. Given the high particle count, minimizing inter-iteration data movement was essential to avoid PCIe bandwidth becoming a limiting factor. Since we expected from the outset to be limited by data transfers, we employed two asynchronous CUDA streams: one dedicated to computation and another exclusively for memory transfers between devices. While the use of asynchronous streams increases effective bandwidth utilization, it does not fully solve the problem, as certain synchronizations between streams are required to ensure correctness and prevent unsafe overlaps between computation and data movement.

In line with the strategy of minimizing data transfers between iterations, only the magnetic and electric fields are transferred from the CPU to the GPU at each iteration, together with the cell index of each particle buffer. At the beginning of each iteration, the current buffers are reset to zero using `cudaMemset`, and the updated current values are transferred back to the CPU only at the end of the iteration.

In practice, refining which data are transferred led to only marginal improvements in execution time, primarily due to the large number of iterations in the simulation. We could fix this problem by changing the full application to CUDA, so all data resides in GPU memory. Since the hotspot only specified optimizing the *spec\_advance*, we decided not to port the other parts to the GPU.

In our implementation, we defined a *gpu\_context\_t* structure that encapsulates the simulation state exchanged between host and device. This structure includes all particle buffers for every species, as well as the current and electromagnetic field buffers. Using this abstraction, we maintain two instances of the context: *h\_gpu\_ctx* on the host and *d\_gpu\_ctx* on the device. In this way, the code itself becomes more reproducible for future implementations and more modular, separating both interfaces.

At the kernel level, one of the main limitations arises from the use of atomic operations during the current deposition phase. Although shared memory could, in principle, be used to reduce global atomics [4], this approach is not effective in our case because particles are not sorted by cell. As a result, per-block shared memory accumulation would not significantly

increase cell reuse, limiting its potential benefits. Since particle sorting was outside the scope of this work, we still used a global atomic update strategy for current deposition.

In contrast, the computation of the total particle energy is well suited for parallel reduction. We therefore implemented a reduction strategy using both warp-level and block-level reductions. To achieve this we created a buffer, *block\_energies*, which stores the partially reduced energy value produced by each block after the main kernel execution.

Within each block, multiple warps may be active. Each warp first reduces its partial energy using warp-level reduction based on register operations and warp shuffle instructions, which are faster than shared or global memory accesses. The resulting per-warp values are then further reduced by a single warp to produce one energy value per block.

After the main kernel completes, a secondary kernel is launched to reduce the contents of *block\_energies* using the same hierarchical reduction strategy. This warp-shuffle-based approach significantly improves performance compared to a reduction based on global atomic operations.

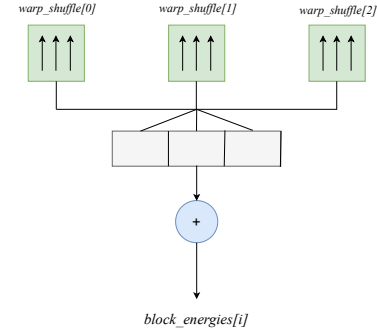


Fig. 6. This figure shows the reduction level strategy at the warp level for a block.

#### 1) Kernel configuration and occupancy considerations

The impact of block size on the performance of the *spec\_advance* kernel was evaluated by measuring total execution time for different numbers of threads per block. The results are shown in Figure 7.

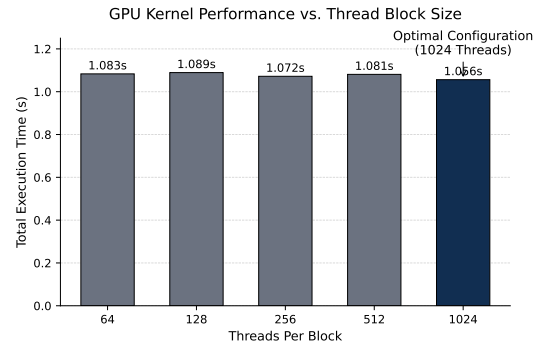


Fig. 7. GPU kernel execution time as a function of the number of threads per block on the NVIDIA A100.



As shown in Figure 7, the lowest execution time is achieved with 1024 threads per block, but the improvement over the 256-thread configuration is marginal (1.5%) and within typical run-to-run variability. From an architectural perspective, 256 threads per block already provide sufficient occupancy on the NVIDIA A100 to hide memory latency, and increasing block size does not yield meaningful additional parallelism due to global memory access and atomic contention during current deposition.

Furthermore, larger blocks increase register pressure and reduce scheduling flexibility, which may negatively impact robustness and performance portability. For these reasons, 256 threads per block were selected as the baseline configuration for subsequent experiments.

## 2) Weak scalability on GPU

Weak scalability of the CUDA implementation was evaluated by proportionally increasing the problem size while keeping a fixed kernel configuration of 256 threads per block. The number of particles per cell (PPC) was increased from 125 to 8000 (16 $\times$ ), with results shown in Figure 8.

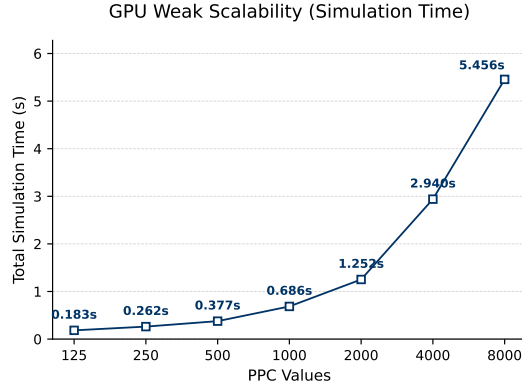


Fig. 8. Weak scalability of the CUDA implementation on the NVIDIA A100 for increasing particles per cell (PPC), using a fixed block size of 256 threads.

As the workload scales, execution time increases from 0.38 s to 5.45 s, deviating from ideal weak scalability. This degradation is driven by increasing atomic contention in the current deposition phase, which becomes increasingly serialized as particle count grows.

Since kernel occupancy is already saturated, larger problem sizes do not improve latency hiding. Memory traffic and atomic operations dominate execution, resulting in near-linear runtime growth. Improving weak scalability would require algorithmic changes, such as alternative deposition schemes or a fully GPU-resident execution model.

## D. Cross-architecture performance comparison

Figure 9 compares the best execution-time configuration for each architecture. The AMD EPYC 7742 with OpenMP (64 cores) achieves the lowest runtime (0.145 s), followed by the Fujitsu A64FX with 48 threads (0.494 s), benefiting from its high-bandwidth HBM2 memory subsystem.

The NVIDIA A100 GPU reaches approximately 1.07 s, with performance limited by atomic contention during current deposition and host-device synchronization overheads. MPI-based configurations on both CPU architectures perform significantly worse ( $\sim 4$  s), as communication costs dominate for this problem size.

Overall, shared-memory CPU parallelism delivers the best performance for the current ZPIC design, while GPU and MPI approaches are constrained by communication and synchronization bottlenecks.

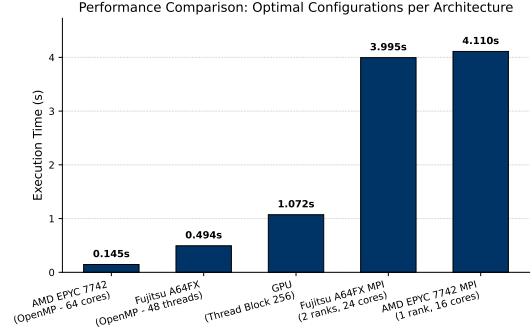


Fig. 9. Cross-Architecture Performance Comparison

## VI. CONCLUSIONS AND FUTURE WORK

This work evaluated the performance of ZPIC on multiple architectures and parallel programming models on the Deucalion supercomputer, using two workloads with distinct computational characteristics. Results show that PIC performance is strongly workload-dependent and primarily limited by memory bandwidth and communication overheads rather than peak compute capability.

In the laser-particle case (WA1), grid-based kernels with low arithmetic intensity dominate execution. On the Fujitsu A64FX, data-layout transformations and SVE vectorization significantly improved single-core performance, but strong scalability was quickly limited by HBM2 bandwidth saturation. In the two-stream case (WA2), the particle pusher (`spec_advance`) dominates runtime, enabling better scalability. The AMD EPYC 7742 achieved the lowest execution times, while the A64FX provided competitive performance with higher energy efficiency at moderate thread counts.

Hybrid MPI+OpenMP enabled multi-node execution but scaled poorly due to particle redistribution and collective communication overheads. CUDA acceleration on the NVIDIA A100 delivered speedups for moderate problem sizes, yet performance was constrained by atomic contention in current deposition and by host-device data transfers.

Overall, shared-memory CPU parallelism is the most effective approach for the current ZPIC design. Future work should prioritize algorithmic changes, including spatial domain decomposition, improved particle sorting and deposition schemes, and fully GPU-resident implementations, as well as extensions to multi-GPU and heterogeneous performance modeling.

## REFERENCES

- [1] A. Beck *et al.*, “Adaptive SIMD optimizations in particle-in-cell codes with fine-grain particle sorting,” *Computer Physics Communications*, vol. 228, pp. 1–14, 2018. Available: <https://doi.org/10.1016/j.cpc.2019.05.001>
- [2] R. Okazaki *et al.*, “Supercomputer Fugaku CPU A64FX Realizing High Performance and Energy Efficiency,” *Fujitsu Technical Review*, vol. 56, no. 1, pp. 15–23, 2020. Available: <https://www.fujitsu.com/global/documents/about/resources/publications/technicalreview/2020-03/article03.pdf>
- [3] NVIDIA, *CUDA C++ Programming Guide: Asynchronous Execution*, Online documentation, 2025. Available: <https://docs.nvidia.com/cuda/>
- [4] NVIDIA, *NVIDIA A100 Tensor Core GPU Architecture*, White Paper, 2020. Available: <https://www.nvidia.com/en-us/data-center/ampere-architecture/>
- [5] Minho Advanced Computing Center (MACC), *Deucalion – Architecture*, Online documentation. Available: <https://docs.macc.fccn.pt/deucalion/>

## APPENDIX

This appendix reports the OpenMP strong scalability results of ZPIC on the Fujitsu A64FX architecture. Results are shown relative to the SVE-optimized single-thread baseline. These measurements correspond to the WA1 configuration and were obtained prior to the modifications introduced for the WA2 (two-stream) simulation.

### A. Execution Time

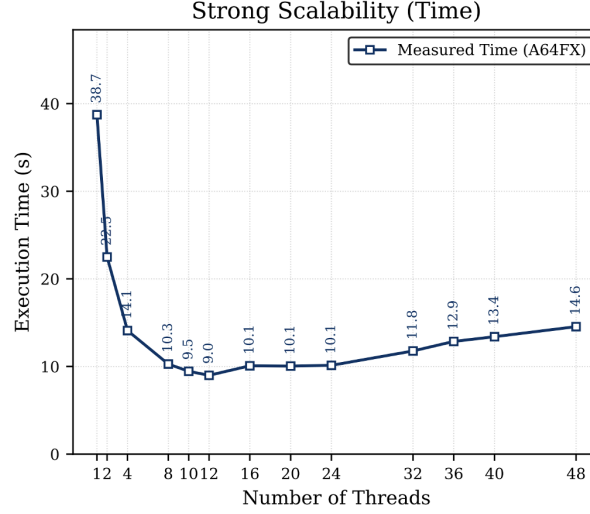


Fig. 10. Strong scalability of ZPIC on the Fujitsu A64FX. Performance improves up to 12 threads (one CMG), after which scalability degrades due to HBM2 bandwidth saturation and increased OpenMP overhead.

### B. Speedup

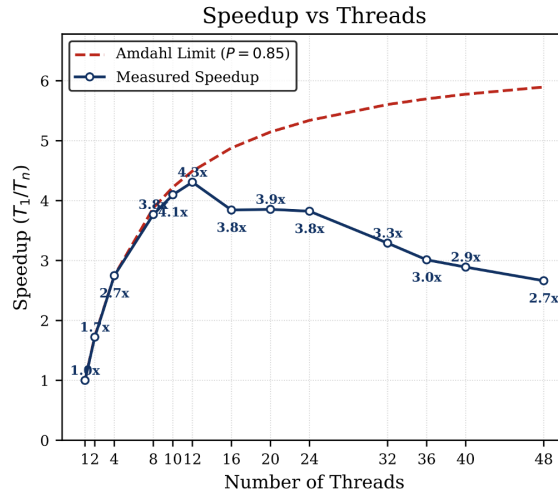


Fig. 11. Speedup relative to the SVE-optimized single-thread baseline on the Fujitsu A64FX. A maximum speedup of  $4.31\times$  is achieved at 12 threads, after which memory bandwidth saturation limits further gains.



This appendix presents detailed strong scalability results for the WA2 (two-stream instability) configuration on both CPU architectures.

### C. Fujitsu A64FX

Performance improves rapidly up to 12 threads, corresponding to execution within a single CMG, and continues to improve at a slower rate as additional CMGs are utilized. Deviations from ideal scaling at higher thread counts reflect residual serial regions and memory bandwidth limitations, consistent with Amdahl’s Law.

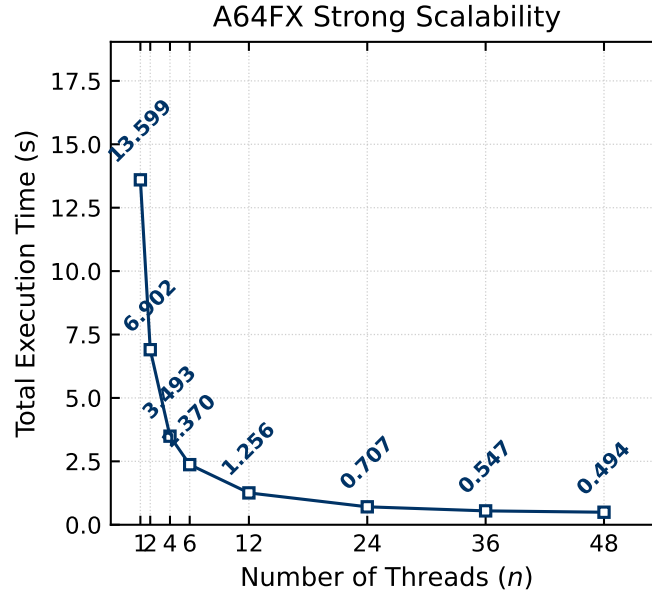


Fig. 12. Strong scalability of the ZPIC two-stream simulation on the Fujitsu A64FX, showing total execution time as a function of the number of threads.

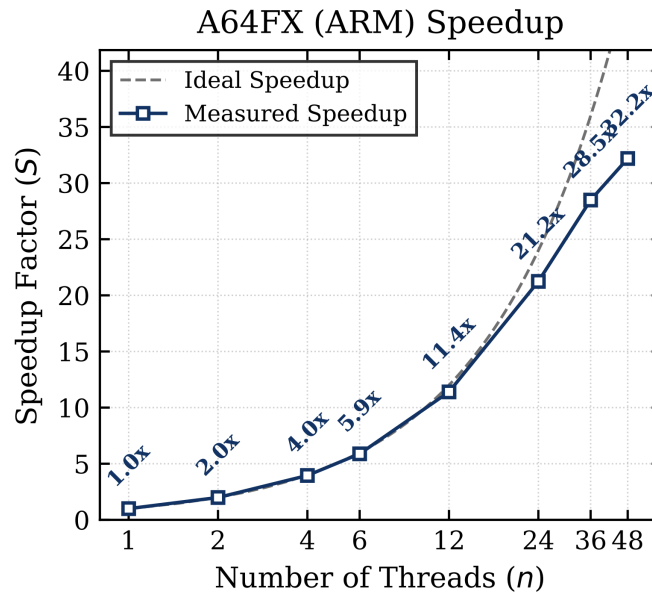


Fig. 13. Measured strong-scaling speedup of the ZPIC two-stream simulation on the Fujitsu A64FX compared to the ideal linear speedup.

### D. AMD EPYC 7742

Due to the particle-dominated nature of the workload, the application benefits from the large core count of this platform. Execution time decreases steadily up to 64 threads, after which NUMA effects and increased memory pressure limit scalability.

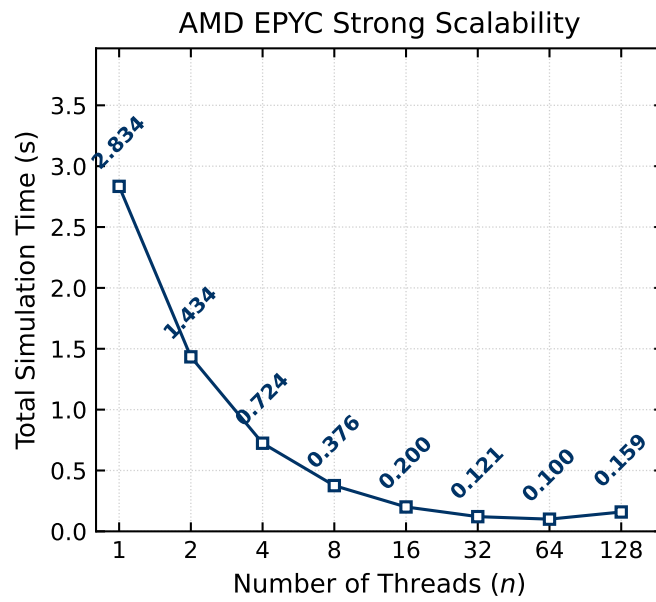


Fig. 14. Strong scalability of the ZPIC two-stream simulation on the AMD EPYC 7742, showing total execution time as a function of the number of threads.

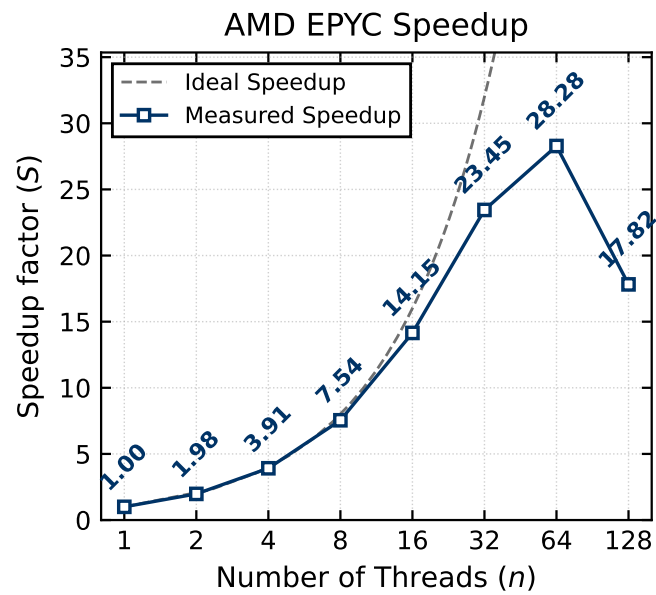


Fig. 15. Measured strong-scaling speedup of the ZPIC two-stream simulation on the AMD EPYC 7742 compared to the ideal linear speedup.

This appendix reports the strong and weak scalability of the hybrid MPI+OpenMP implementation for the WA2 configuration on both CPU architectures.

MPI ranks were placed to preserve architectural locality: CMG-aligned placement on A64FX (12 threads/rank) and CCD/NUMA-aware placement on AMD EPYC (16 threads/rank). Scaling was achieved by increasing the number of MPI ranks across nodes.

#### E. Strong Scalability

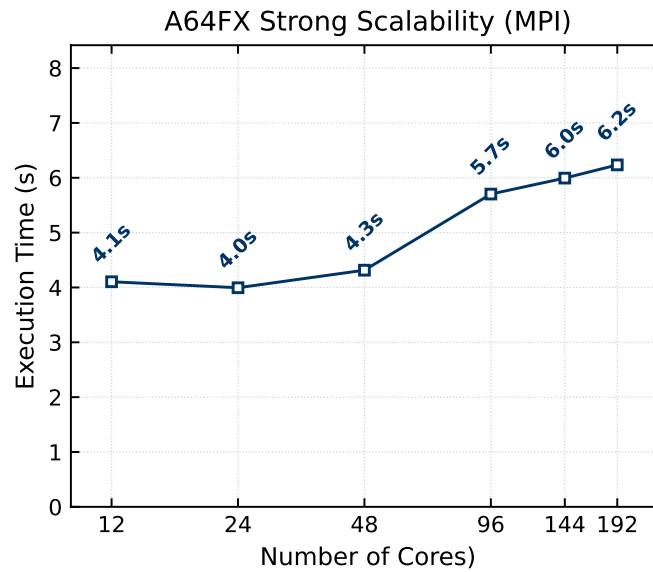


Fig. 16. Strong scalability of the WA2 MPI implementation on Fujitsu A64FX. Performance saturates quickly and degrades across nodes due to particle redistribution and collective communication overheads.

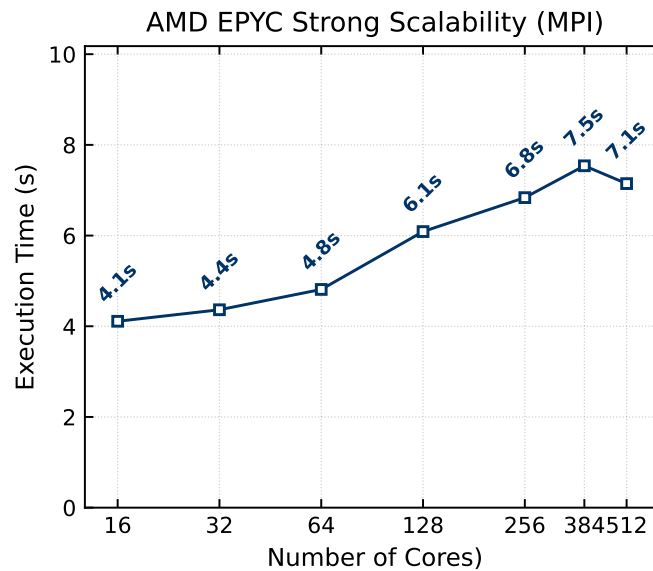


Fig. 17. Strong scalability of the WA2 MPI implementation on AMD EPYC 7742. Scaling degrades at higher core counts as communication and synchronization dominate execution time.

## F. Weak Scalability

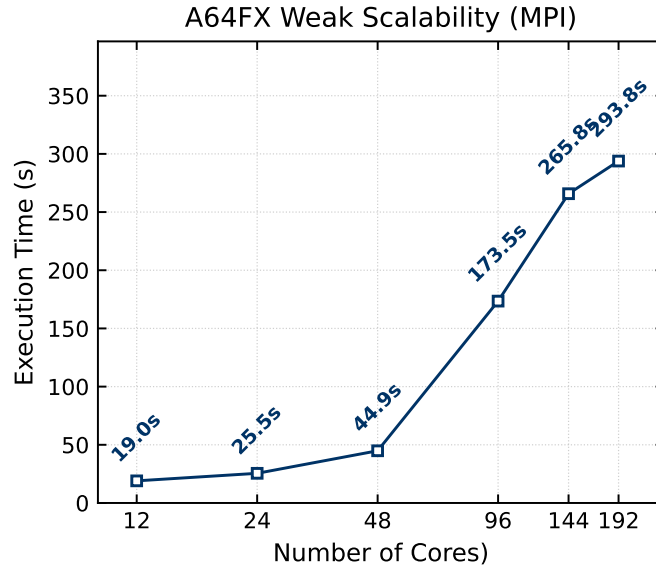


Fig. 18. Weak scalability of the WA2 MPI implementation on Fujitsu A64FX. Execution time increases with scale due to growing collective communication and synchronization costs.

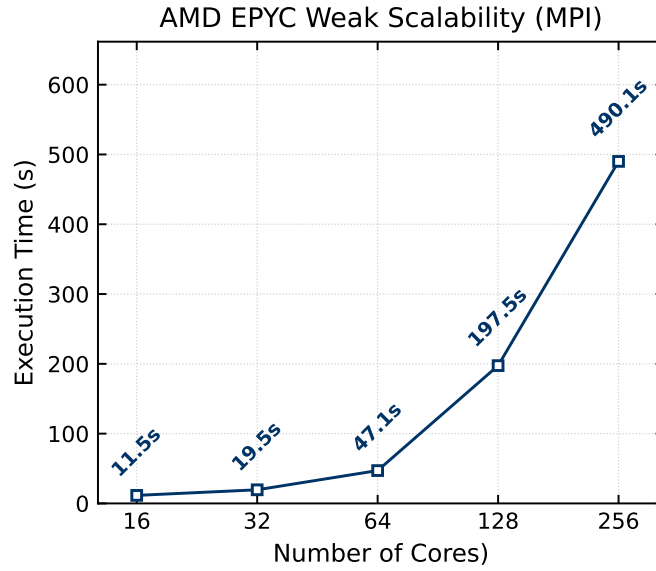


Fig. 19. Weak scalability of the WA2 MPI implementation on AMD EPYC 7742. Despite NUMA-aware placement, execution time increases significantly at larger scales.

## G. Summary

The MPI scalability results confirm that while the hybrid MPI+OpenMP approach enables multi-node execution, scalability is fundamentally limited by the particle-based decomposition. Iteration-level particle redistribution and global reductions dominate execution time at scale, motivating GPU acceleration as the primary optimization strategy for the WA2 configuration.